Here is the practical, step-by-step implementation guide to building the **MCP Layer**, the **Agentic Orchestration Layer**, and integrating them together.

Phase 1: Build the MCP Layer

Step 1: Set Up the Custom FastMCP Server

- **Objective:** Wrap your existing agronomy database, satellite indices, and yield models into standard MCP tools.
-
- **Actions:**
-

1. Install the official Python MCP library (`pip install "mcp[cli]"` or use FastMCP).
2.
3. Create a server file (e.g., `agrimind_mcp.py`) exposing the core tools:
4.
    - `get_plot_status(plot_id: str)`: Queries your database for the plot's polygon, crop type, planting date, and current phenological stage.
    -
    - `analyze_crop_telemetry(plot_id: str)`: Pulls Sentinel NDVI/NDRE, CHIRPS rainfall anomaly, soil moisture, and your ML yield prediction. Returns structured JSON containing telemetry values with attached timestamps and unique `evidence_id` tags (e.g., `EVID-RAIN-01`).
    -
    - `evaluate_calendar_window(crop_stage: str, rain_anomaly: float, spray_risk: bool)`: Computes deterministic checks on whether the environmental conditions support planting, spraying, or holding.
    -
    - `draft_advisory(plot_id: str, action: str, evidence_ids: list[str], rationale: str)`: The required stateful/action tool. Saves a structured draft in your database marked with status `PENDING_APPROVAL`.
    -
5. Ensure the script runs over standard input/output (`mcp.run()`), which allows orchestrators to connect via child-process `stdio` pipes.

Step 2: Configure the Borrowed 3rd-Party MCP Server

- **Objective:** Meet the hackathon requirement of using an existing, maintained MCP server without writing redundant code.
-
- **Actions:**
-

1. Pull the official SQLite MCP server (or an official Postgres/filesystem MCP).
2.
3. Configure it to point directly to your household plot records or historical season archives.
4.
5. Verify it exposes standard read tools (e.g., `read_query`) so the agent can discover past yield trends without custom SQL wrappers.
6.

Phase 2: Build the Agentic Layer (LangGraph + Open-Weights LLM)

Step 1: Set Up the Open-Weights Inference Engine

- **Objective:** Ensure data sovereignty and meet open-weights criteria.
-
- **Actions:**
-

1. Pull an open-weights model. (hugging face)
2.
3. Connect to the model in Python using `langchain-community` or `ChatOllama` with tool-calling capabilities enabled.
4.

Step 2: Define State Schema & Finite State Graph

- **Objective:** Construct a strict state machine rather than an unbounded conversational loop.
-
- **Actions:**
-

1. Define a TypedDict state (e.g., `AgriState`) tracking `plot_id`, `evidence_store`, `advisory_draft`, `step_count`, `tool_call_count`, and `officer_decision`.
2.
3. Implement runtime budgets: hardcode `MAX_STEPS = 12` and `MAX_TOOL_CALLS = 15`.
4.
5. Wire the graph nodes in sequence:
6.

    - plan_and_fetch: Model determines which MCP tools to call based on the plot ID.
    -
    - reason_action: Model analyzes the returned telemetry and selects PLANT, SPRAY, or HOLD.
    -
    - draft_call: Model calls the MCP draft_advisory action tool with its reasoning and citations.
    -

Step 3: Implement the Dual Guardrails

- **Objective:** Enforce safety and eliminate hallucinations or bypassed reviews.
-
- **Actions:**
-

1. **Deterministic Validator Node:** Write a pure Python condition (no LLM) that checks len(evidence_ids) >= 2. If critical environmental data is missing, route immediately to INSUFFICIENT_EVIDENCE.
2.
3. **Human Approval Interrupt Node:** Use LangGraph's native interrupt() function. When valid evidence exists, freeze execution and save the state checkpoint to disk (e.g., using SqliteSaver).
4.
5. **Terminal Nodes:**
6.

    - Route an officer rejection to REJECTED.
    -
    - Route an officer approval to SIMULATED_SMS_PREPARED.
    -

Phase 3: Integration (Connecting Agent to MCP)

Step 1: Establish the Client-Server Bridge

- **Objective:** Allow LangGraph to dynamically discover and execute the MCP tools.
-
- **Actions:**
-

1. In your orchestrator codebase, initialize the mcp client session using standard process pipes (StdioServerParameters):
2.

    - Point process 1 to your Python FastMCP script (python agrimind_mcp.py).

    -
    - Point process 2 to the borrowed SQLite MCP server.
    -
3. Call `session.list_tools()` at startup. Convert the returned JSON schemas into callable LangChain/LangGraph tools so the Qwen model knows the exact function signatures.
4.

Step 2: Implement Deduplication and Circuit Breaking

- **Objective:** Keep execution within the hackathon evaluation limits.
-
- **Actions:**
-

1. Add an in-memory dictionary cache on tool calls: if the agent requests `analyze_crop_telemetry("KE-001")` twice, return the cached result instead of hitting Google Earth Engine or your database again.
2.
3. Implement safe retry logic: retry transient read timeouts up to 2 times, then safely exit if the service remains unreachable.
4.

Step 3: Wire the Officer Review Endpoint

- **Objective:** Connect the human-in-the-loop interface to the paused LangGraph thread.
-
- **Actions:**
-

1. Expose an endpoint or CLI command: `POST /api/advisories/{thread_id}/decide`.
2.
3. When the extension officer clicks "Approve" or "Reject" on their UI/CLI, pass the signed payload (including `officer_name`, `timestamp`, and `verdict`) into `graph.resume()`.
4.
5. Verify that resuming the graph writes an audit record and executes the final terminal state.